

Execution (Research Perspective)

VWAP/TWAP/POV, Implementation Shortfall, Market Impact & Execution Scheduling

Converting a Target Position Into Trades Without Moving the Market Against Yourself

A Level 3 module in the Quantitative Finance Curriculum

Covers: VWAP/TWAP/POV · implementation shortfall · arrival price · temporary/permanent market impact · Almgren-Chriss (awareness) · execution scheduling · liquidity-aware execution · order slicing · passive vs aggressive execution · smart order routing (awareness)

Table of Contents

How to Use This Guide

What You Need to Know Before Starting

Understand Before Proceeding

Why It Matters

How This Connects to the Rest of the Curriculum

Glossary of Recurring Terms

Part 1 — Implementation Shortfall & Arrival Price

- 1.1 The Gap Between Research and Reality
- 1.2 Decomposing Implementation Shortfall
- 1.3 Arrival Price as the Execution Benchmark

Part 2 — Market Impact: Temporary and Permanent

- 2.1 Why Trading Moves Prices
- 2.2 Temporary vs. Permanent Impact
- 2.3 Impact Is a Function of Urgency

Part 3 — Execution Scheduling: VWAP, TWAP & POV

- 3.1 TWAP — Time-Weighted Average Price
- 3.2 VWAP — Volume-Weighted Average Price
- 3.3 POV — Percentage of Volume
- 3.4 Choosing Between Them

Part 4 — Order Slicing & Liquidity-Aware Execution

- 4.1 Why a Single Order Becomes Many Orders
- 4.2 Liquidity-Aware Slicing
- 4.3 Passive vs. Aggressive Execution

Part 5 — Almgren-Chriss and the Cost/Risk Trade-Off (Awareness)

- 5.1 Framing Execution as an Optimisation
- 5.2 The Almgren-Chriss Trajectory
- 5.3 Smart Order Routing (Awareness)

Part 6 — Execution in the Research Workflow

- 6.1 From Flat Cost Assumption to Real Execution Model
- 6.2 Building Execution Costs Into a Backtest

Practical Exercise

Common Beginner Mistakes

Advanced Extensions

Professional Applications

Where to Go Next

How to Use This Guide

This document is a companion to Execution (Research Perspective), the third-level module in the quantitative finance curriculum. It covers the problem of converting a target position into a sequence of real trades without moving the market against yourself — the step that determines how much of a signal's theoretical edge actually survives contact with the market.

The module is written from the research side of execution: what a researcher needs to understand about execution to build realistic backtests and interpret live trading results, rather than the full desk-level implementation covered later in the Trading track. Every concept is paired with a worked example, a short Python sketch, and a chart wherever a picture makes the mechanics clearer than a formula alone.

What You Need to Know Before Starting

- **Market Microstructure (Level 2)** — the mechanics of order books, bid-ask spreads, and how orders interact with resting liquidity; this module assumes you already know what a limit order, a market order, and the order book look like.
- **Backtesting (transaction cost modelling)** — familiarity with how a backtest applies a cost assumption to simulated trades; this module explains where that cost assumption should actually come from.

Understand Before Proceeding

Prerequisite gate —

Do not treat execution as a detail to bolt onto a strategy after the research is finished. A signal is evaluated in research against a theoretical fill price — often the mid-price at the moment of the trading decision. Every basis point between that theoretical price and the price you actually trade at is edge that never reaches the P&L, and for high-turnover or large-size strategies that gap can consume most, or all, of the strategy's apparent advantage.

Why It Matters

Research signals are evaluated at a theoretical fill price; execution determines how much of that theoretical edge survives contact with the market. A signal that shows a 2% annualized edge in a backtest that assumes costless, instant fills might show a materially smaller edge — or none at all — once realistic execution costs are applied. Understanding execution is what separates a researcher who can tell the difference between a strategy that is genuinely unprofitable and one that is simply being executed badly.

How This Connects to the Rest of the Curriculum

- **Research** — Execution-cost assumptions in a backtest should be grounded in real execution-algorithm behaviour, not a flat cost estimate; a backtest that assumes a single constant spread cost per trade, regardless of order size or urgency, will systematically misprice large or urgent trades.
- **Development** — Building an execution simulator or connecting to a broker's execution algorithms is core Level 4/5 infrastructure work — translating the concepts in this module into production code that can actually route and monitor live orders.
- **Trading** — This is the trading track's core subject matter at Level 5 — introduced here from the research side, so that by the time a researcher moves into (or works alongside) trading, the vocabulary and intuition are already familiar.

Glossary of Recurring Terms

Arrival price — The market price (typically the mid-price) at the moment an order is submitted for execution, or the trading decision is made; the standard benchmark against which execution quality is measured.

Implementation shortfall — The total difference between the price at the moment of the investment decision (decision price) and the actual realised price of the executed trade, including delay cost, market impact, and any opportunity cost from unfilled quantity.

Market impact — The amount by which a trade moves the market price against the trader, split into a temporary component (which decays) and a permanent component (which persists).

Temporary impact — The portion of market impact caused by short-term liquidity consumption that reverts once trading pressure stops — compensation liquidity providers demand for absorbing the order.

Permanent impact — The portion of market impact that persists after trading stops, reflecting the market's updated belief about the asset's fair value given the information content of the trade.

VWAP (Volume-Weighted Average Price) — Both a benchmark (the volume-weighted average price over a period) and an execution strategy that slices an order in proportion to expected or real-time market volume.

TWAP (Time-Weighted Average Price) — Both a benchmark (the simple average price over a period) and an execution strategy that slices an order evenly across fixed time intervals, regardless of market volume.

POV (Percentage of Volume) — An execution strategy that continuously trades a fixed percentage of real-time market volume, speeding up and slowing down as the market's own trading activity changes.

Order slicing — Breaking a large parent order into many smaller child orders executed over time, to reduce the market impact any single trade would otherwise cause.

Participation rate — The fraction of total market volume that an order (or execution algorithm) represents over a given period; a direct proxy for how aggressively, or urgently, an order is being worked.

Passive execution — Working an order primarily with resting limit orders that provide liquidity rather than take it, typically earning the spread but risking non-execution if the market moves away.

Aggressive execution — Working an order primarily by taking liquidity (crossing the spread with marketable orders), guaranteeing faster fills at the cost of paying the spread and more market impact.

Almgren-Chriss model — A framework that formalizes optimal execution as a trade-off between market impact cost (minimized by trading slowly) and timing risk (minimized by trading quickly), analogous to mean-variance optimisation applied to a single order's trading trajectory.

Smart order routing (SOR) — Automated logic that decides, order by order, which venue (exchange, dark pool, or other liquidity source) to route each child order to, in order to minimize cost and maximize fill probability across a fragmented market.

Part 1 — Implementation Shortfall & Arrival Price

Before looking at any specific execution technique, it helps to have a precise way of measuring execution quality in the first place. Implementation shortfall is the standard framework for doing exactly that: it accounts for every source of slippage between the moment a decision is made and the moment a trade is actually completed.

1.1 The Gap Between Research and Reality

A backtest typically assumes a strategy trades at, or very near, the price observed at the moment a signal fires — often the mid-price of the bid-ask spread. In live trading, that price is never quite achievable: time passes between decision and order submission, the order itself consumes available liquidity and pushes price against the trader, and sometimes part of the intended order simply cannot be filled at all before the opportunity passes. Implementation shortfall is the metric that captures all of this in one number.

1.2 Decomposing Implementation Shortfall

Implementation shortfall is defined as the difference between the realised (actual) execution price and the decision price, and is conventionally decomposed into three components:

$$\text{Implementation Shortfall} = \text{Delay Cost} + \text{Market Impact} + \text{Opportunity Cost}$$

- **Delay cost** — price drift between the moment the decision is made and the moment the order actually begins executing (e.g. time spent routing the order, or waiting for a trading window).
- **Market impact (trading cost)** — the price movement directly caused by the act of trading itself, covered in depth in Part 2.
- **Opportunity cost** — the cost of any portion of the order that is never filled at all, valued at the price the position would have been worth had it been completed.

Implementation Shortfall: From Decision Price to Realized Price

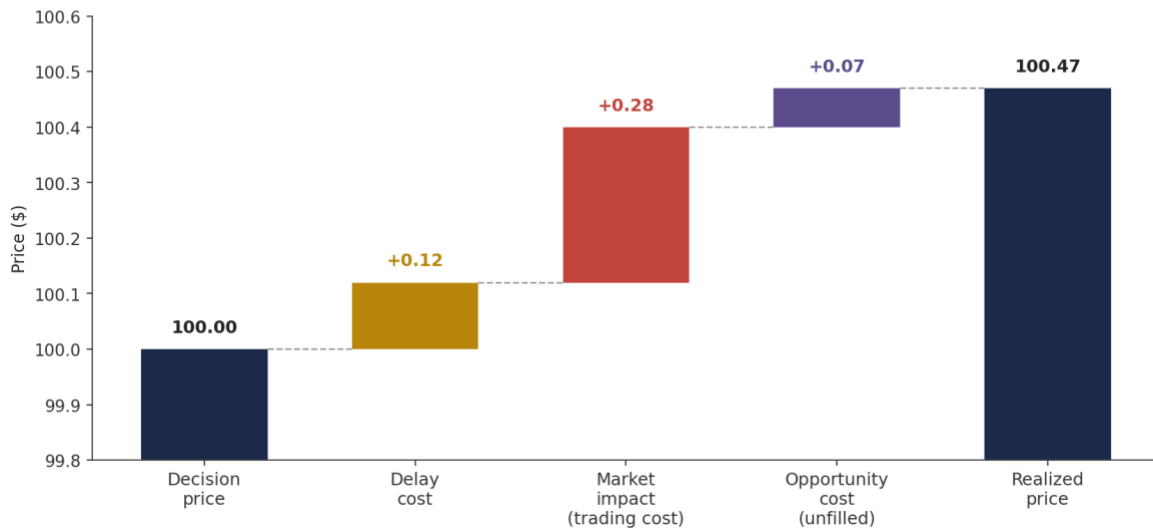


Figure 1.1 — Implementation shortfall walks the decision price forward through each source of slippage to the realised price. A signal's research-stage edge is measured against the decision price; every increment shown here is edge that never reaches the P&L.

```
decision_price = 100.00
delay_cost = 0.12      # price drift before the order starts working
market_impact = 0.28   # cost directly caused by trading
opportunity_cost = 0.07 # value of quantity never filled

realized_price = decision_price + delay_cost + market_impact + opportunity_cost
implementation_shortfall = realized_price - decision_price # = 0.47, i.e. 47 bps on a $100 stock
```

1.3 Arrival Price as the Execution Benchmark

In practice, "decision price" is hard to timestamp precisely (when exactly was the decision made?), so most execution desks benchmark against arrival price instead: the market price at the moment the order is actually submitted for execution. Comparing the realised execution price to arrival price isolates market impact and delay within the execution process itself, separate from any earlier research-to-order-submission lag, and is the most common metric reported in a transaction cost analysis (TCA) report.

Tip —

When comparing execution algorithms, always confirm which benchmark is being used. "Beat VWAP by 3 bps" and "beat arrival price by 3 bps" are different, non-comparable claims: VWAP is a full-period average that a large order can itself distort, while arrival price is a fixed point in time set before the order started trading.

Part 2 — Market Impact: Temporary and Permanent

Market impact — the price movement caused directly by the act of trading — is usually the largest component of implementation shortfall for any order of meaningful size. Understanding its structure is central to understanding why execution algorithms are built the way they are.

2.1 Why Trading Moves Prices

A buy order consumes resting sell-side liquidity in the order book; once the nearest price levels are exhausted, the order must reach further up the book, executing at progressively worse prices. Beyond this purely mechanical effect, other market participants also observe the trading activity and update their own quotes — a large, persistent buyer signals information (or at least demand) that other participants rationally price in.

2.2 Temporary vs. Permanent Impact

It is useful to separate market impact into two components with very different implications for execution strategy:

- **Temporary impact** — the portion of the price move that reverts once trading pressure stops. This is compensation that liquidity providers demand for absorbing an order quickly, and it disappears (mostly) once the order is done.
- **Permanent impact** — the portion of the price move that persists. This reflects the market's updated view of fair value given the information content of the trade, and does not revert.

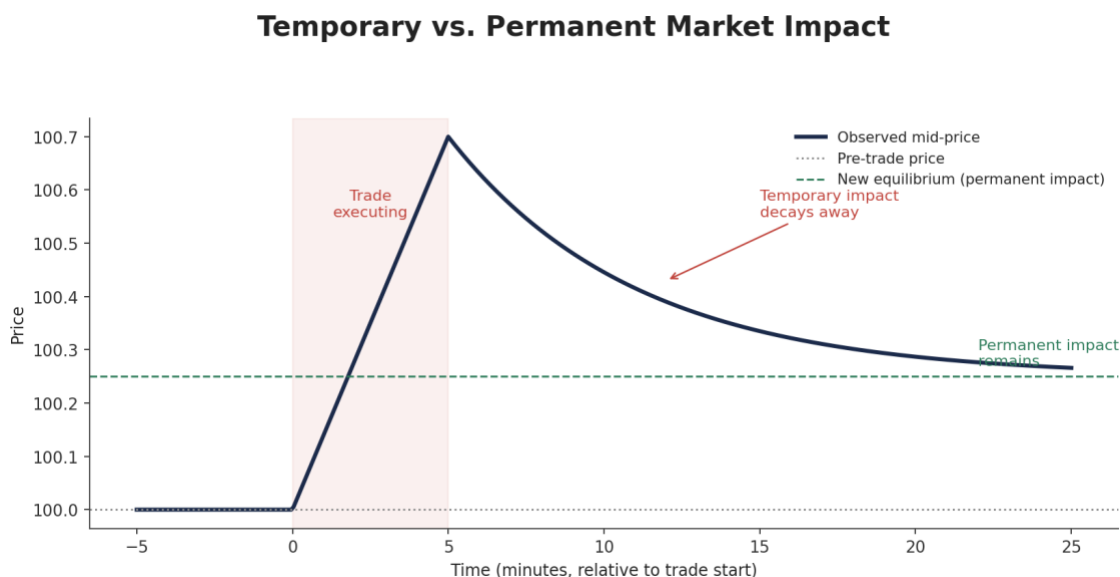


Figure 2.1 — During execution, price moves against the trader. Once trading stops, the temporary component decays away while the permanent component remains as the new equilibrium price.

This distinction matters directly for execution strategy: temporary impact is a cost that scales with how quickly liquidity is consumed, so it can be reduced by trading more slowly. Permanent impact reflects information leakage and is far less controllable by execution speed alone — it is closer to an unavoidable cost of the trade's size relative to the market, no matter how it is worked.

2.3 Impact Is a Function of Urgency

A common early misconception is treating market impact as a fixed cost per trade. In reality, impact cost is a strongly increasing function of trading speed (equivalently, participation rate): the faster an order is worked relative to normal market volume, the more liquidity it must consume per unit time, and the worse the average fill price becomes. This relationship is typically modelled as super-linear (impact cost grows faster than proportionally with participation rate).

Faster Execution Costs More: Impact Cost vs. Participation Rate

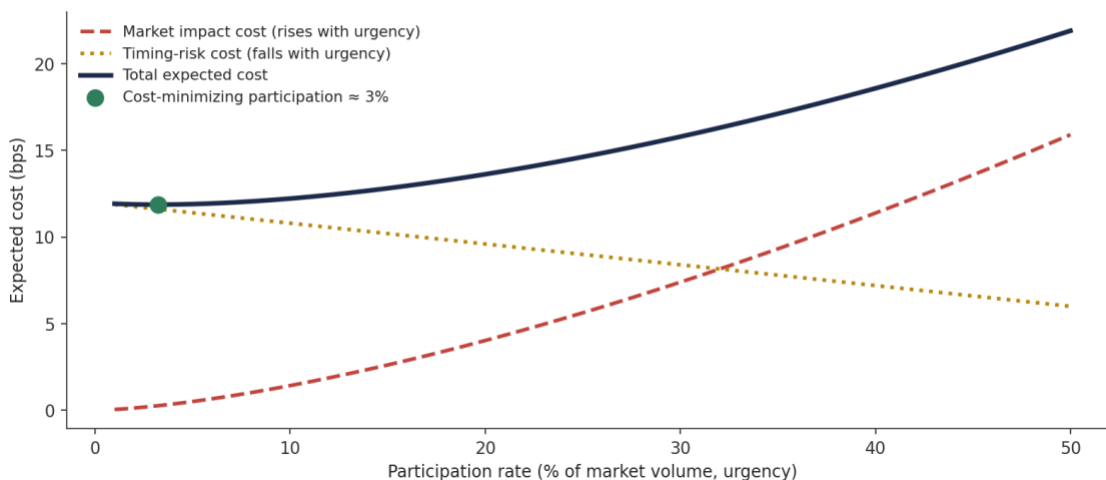


Figure 2.2 — Market impact cost rises steeply with participation rate (urgency), while the cost of remaining exposed to adverse price drift (timing risk) falls as an order is completed faster. The optimal participation rate balances the two, and shifts with how urgent the order genuinely is.

Common beginner mistake —

Ignoring that market impact is itself a function of urgency — faster execution costs more. A cost model that applies the same per-share cost regardless of how quickly an order needs to be completed will badly misprice both urgent orders (underestimating their true cost) and patient ones (overestimating it).

Part 3 — Execution Scheduling: VWAP, TWAP & POV

Given that impact rises with trading speed, the natural response is to spread a large order out over time rather than trading it all at once. The three most common scheduling algorithms — TWAP, VWAP, and POV — differ in exactly how they spread that order out.

3.1 TWAP — Time-Weighted Average Price

TWAP slices an order into equal-sized child orders spread evenly across a fixed time window, regardless of how much the market itself is trading at any given moment. It is simple, predictable, and a reasonable default

when no strong view exists on the intraday volume pattern — but it can leave an order trading a disproportionately large share of volume during naturally quiet periods.

$$\text{Child order size at each interval} = \text{Total order size} / \text{Number of intervals}$$

3.2 VWAP — Volume-Weighted Average Price

VWAP instead slices the order in proportion to expected market volume at each point in the trading day, trading more heavily during naturally high-volume periods (typically the open and close) and less during the mid-day lull. The goal is for the order's own participation rate to stay roughly constant relative to the market, so the order blends into natural liquidity rather than standing out from it.

Order Slicing: TWAP, VWAP, and POV Execution Schedules

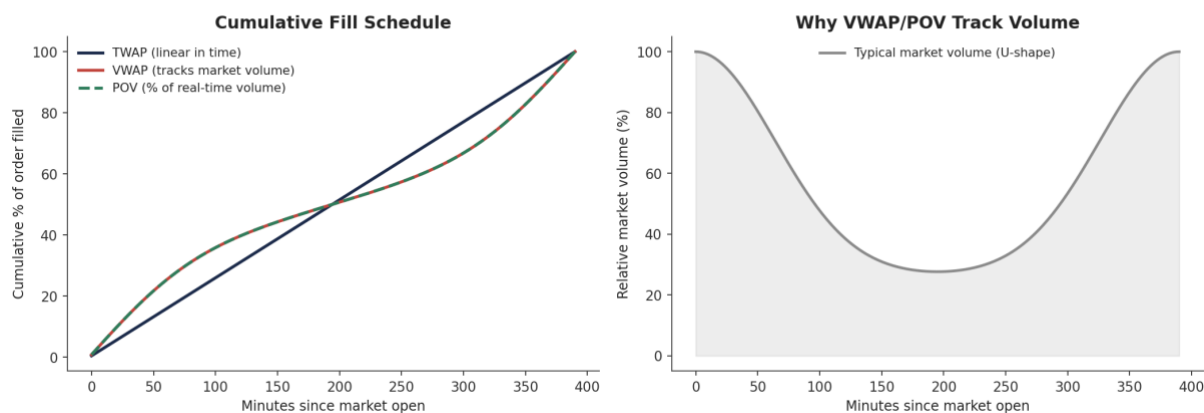


Figure 3.1 — TWAP slices evenly across time; VWAP and POV track the market's characteristic U-shaped intraday volume profile instead, trading more heavily near the open and close.

3.3 POV — Percentage of Volume

POV (also called "participation rate" execution) is the real-time analogue of VWAP: rather than following a pre-set schedule based on historical volume patterns, it continuously monitors actual market volume and trades a fixed target percentage of it as it happens. POV speeds up automatically if the market is trading more heavily than expected, and slows down if it isn't — trading VWAP's forecast-based schedule for direct responsiveness to realised conditions.

```
# Simplified POV logic, evaluated at each time step
target_participation = 0.10          # trade 10% of real-time volume

def next_clip_size(market_volume_this_interval, target_participation):
    return market_volume_this_interval * target_participation

# if the market suddenly trades 3x its usual volume, the POV algorithm
# automatically sends a 3x larger clip in that interval, and vice versa
```

3.4 Choosing Between Them

Algorithm	Schedule basis	Best suited for
TWAP	Fixed, even in time	No strong volume view; simple, predictable orders
VWAP	Historical/forecast volume profile	Matching a VWAP benchmark; liquid, well-behaved names
POV	Real-time market volume	Adapting to unpredictable intraday volume; controlling participation directly

Part 4 — Order Slicing & Liquidity-Aware Execution

TWAP, VWAP, and POV describe how much of an order to trade at each point in time; the remaining question is how each individual child order actually interacts with the order book once it reaches the market.

4.1 Why a Single Order Becomes Many Orders

A large parent order (the total position to be built or unwound) is broken into many smaller child orders — this is order slicing, the mechanical realisation of any of the scheduling algorithms in Part 3. Slicing reduces the market impact any single trade causes, since each child order only needs to consume a small amount of available liquidity rather than sweeping through many price levels at once.

TWAP vs. a Naive Single-Clip Market Order

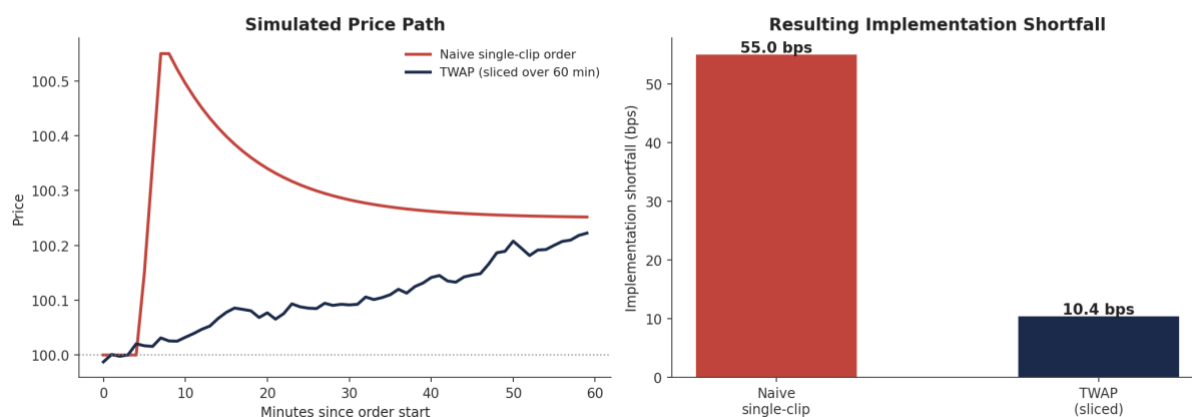


Figure 4.1 — Dumping a full order at once (red) causes a sharp price move, much of which later reverts as temporary impact. Slicing the identical order into small TWAP clips (navy) causes far smaller, more gradual impact and materially lower implementation shortfall.

4.2 Liquidity-Aware Slicing

Beyond simple time- or volume-based schedules, more sophisticated slicing looks directly at real-time order book depth: sizing each child order relative to the liquidity actually resting at the best price levels, rather than a fixed schedule alone. A liquidity-aware algorithm might, for example, opportunistically increase clip size when the book is unusually deep, and pull back sharply when it thins out — reducing impact further than a purely time- or volume-scheduled approach can achieve on its own.

4.3 Passive vs. Aggressive Execution

Each child order also has to decide how to interact with the book: passively (as a resting limit order that provides liquidity) or aggressively (as a marketable order that crosses the spread and takes liquidity).

- **Passive execution** — rests a limit order at or near the best bid/offer, typically earning (rather than paying) the spread if filled, but risking that the market moves away before the order is executed at all — realising the opportunity cost from Section 1.2.

- **Aggressive execution** — crosses the spread to take available liquidity immediately, guaranteeing a fast fill at the cost of paying the spread and typically causing more market impact.

The passive/aggressive choice is, at its core, the same urgency trade-off from Section 2.3 playing out order by order: aggressive execution buys certainty and speed at a higher expected cost, while passive execution is cheaper on average but exposes the order to non-execution and adverse price drift while it waits.

Part 5 — Almgren-Chriss and the Cost/Risk Trade-Off (Awareness)

The tension running through every part of this module — trade fast and pay more impact, or trade slow and accept more timing risk — can be formalized directly as an optimisation problem. The Almgren-Chriss framework is the best-known formalization, covered here at an awareness level ahead of its full treatment at Level 5.

5.1 Framing Execution as an Optimisation

Almgren-Chriss treats the choice of an execution trajectory — how much of the order to trade in each small interval between now and a completion deadline — as a problem of minimizing total expected cost plus a penalty for the variance of that cost, exactly mirroring the structure of mean-variance portfolio optimisation:

$$\min E[\text{Cost}] + \lambda \cdot \text{Var}[\text{Cost}]$$

Here $E[\text{Cost}]$ is dominated by market impact (which falls as execution slows down) and $\text{Var}[\text{Cost}]$ is dominated by timing risk — the variance introduced by exposure to random price moves while the order is still being worked (which rises as execution slows down). λ is a risk-aversion parameter set by how much the trader dislikes variance in execution outcomes relative to expected cost.

5.2 The Almgren-Chriss Trajectory

The Almgren-Chriss Trade-Off: Execution Cost vs. Timing Risk

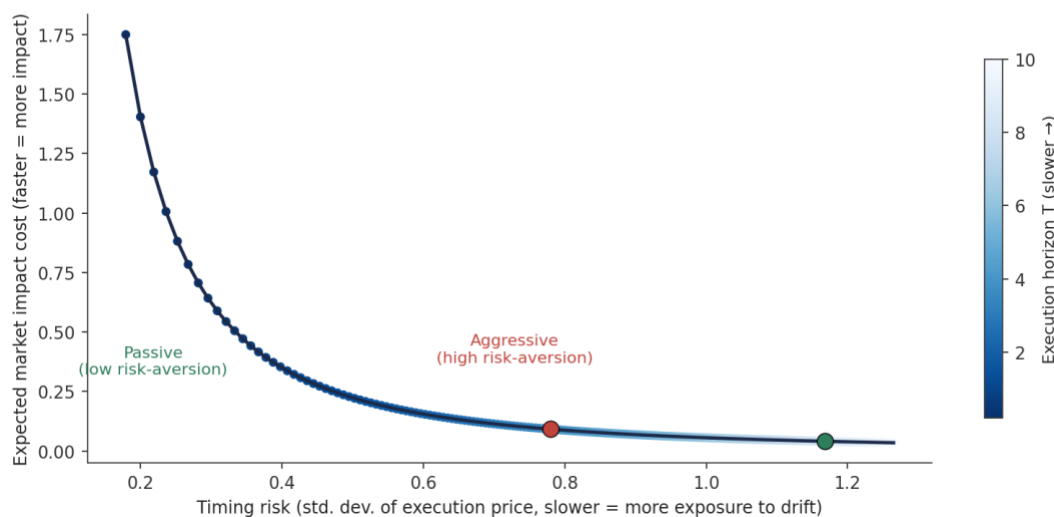


Figure 5.1 — Every point on the curve is an achievable (cost, risk) pair depending on execution speed. A more risk-averse trader (or more urgent order) accepts higher expected impact cost in exchange for lower timing risk; a patient trader accepts more timing risk to reduce impact cost.

Solving the full optimisation yields an optimal trading trajectory — typically front-loaded relative to a flat schedule when risk-aversion is high (trade faster to reduce exposure), and closer to a uniform schedule when

risk-aversion is low. The exact derivation of that trajectory, including the closed-form solution under the model's standard assumptions, is covered in full at Level 5.

5.3 Smart Order Routing (Awareness)

A related, separate problem is not when to trade but where: modern markets are fragmented across many venues — multiple exchanges, alternative trading systems, and dark pools — each with different liquidity, fee structures, and information leakage characteristics. Smart order routing (SOR) is the automated logic that decides, for each child order, which venue (or combination of venues) to send it to, in order to maximize fill probability and minimize cost across that fragmented landscape. Like the full Almgren-Chriss derivation, the mechanics of SOR are a Level 5 topic; the concept to retain here is that scheduling (when/how fast to trade) and routing (where to trade) are distinct problems, both handled by a modern execution algorithm.

Part 6 — Execution in the Research Workflow

The preceding parts describe execution mechanics; this part connects them back to the research process that motivates this module in the first place — building backtests whose cost assumptions actually reflect how a strategy would be traded.

6.1 From Flat Cost Assumption to Real Execution Model

The simplest possible backtest cost model applies a single flat cost per trade — for example, "assume 5 bps of slippage on every trade." This is a reasonable starting point, but it silently assumes that cost does not depend on order size, urgency, or the liquidity of the specific name being traded, all of which Parts 2–5 show to be false. A more realistic backtest models cost as a function of participation rate (Section 2.3), using the same impact-cost curves that a real execution algorithm would face.

Prerequisite gate —

Do not assume a strategy can be executed instantly at the mid-price regardless of order size. This is the single most common source of backtests that look profitable on paper but fail in live trading — the assumption quietly removes exactly the cost that Parts 1–2 of this module exist to describe.

6.2 Building Execution Costs Into a Backtest

A grounded backtest cost model typically combines a spread-crossing cost (roughly constant, tied to the instrument's typical bid-ask spread) with a market-impact term that scales with order size relative to average daily volume, using a functional form calibrated to the impact curves introduced in Section 2.3:

$$\text{Cost (bps)} \approx \text{Spread cost} + k \cdot (\text{Order size} / \text{Average daily volume})^\alpha$$

where k and α (typically greater than 1, reflecting the super-linear relationship from Section 2.3) are calibrated from historical execution data or a broker's own transaction cost analysis (TCA) reports, rather than assumed. Grounding these parameters in actual execution-algorithm behaviour — not a flat estimate — is precisely what separates a backtest that will hold up in live trading from one that will not.

```
import numpy as np

def execution_cost_bps(order_size, avg_daily_volume, spread_bps=2.0, k=8.0, alpha=1.5):
    participation = order_size / avg_daily_volume
    impact_bps = k * participation**alpha
    return spread_bps + impact_bps

# a small order (1% of ADV) costs mostly just the spread;
# a large order (20% of ADV) is dominated by market impact
print(execution_cost_bps(0.01*1e6, 1e6)) # small order
print(execution_cost_bps(0.20*1e6, 1e6)) # large order
```


Practical Exercise

Simulate executing a large order via TWAP vs a naive single-clip market order against a simple market-impact model, and compare implementation shortfall.

- Step 1 — Build a simple market-impact model: temporary impact proportional to trade size relative to available liquidity, decaying exponentially after each trade; permanent impact as a smaller, non-decaying fraction of the same trade size (Section 2.2).
- Step 2 — Simulate a naive execution: submit the entire order as a single market order at time zero, and record the resulting realised price under your impact model.
- Step 3 — Simulate a TWAP execution: slice the same total order into (for example) twenty equal child orders spread evenly over the execution window, applying the same impact model to each child order in sequence.
- Step 4 — Compute implementation shortfall (Section 1.2) for both simulations relative to the same arrival price, and compare — the TWAP execution should show materially lower shortfall, illustrating directly why order slicing exists.

Common Beginner Mistakes

- Assuming a strategy can be executed instantly at the mid-price regardless of order size — ignoring both market impact (Part 2) and the fact that liquidity at the best price is always finite.
- Ignoring that market impact is itself a function of urgency — faster execution costs more (Section 2.3) — and applying a single flat cost estimate to trades of very different sizes and urgency levels.

Advanced Extensions

- **Full Almgren-Chriss optimal execution** — deriving the closed-form optimal trading trajectory under the model's standard assumptions, and extending it to more realistic impact functions (Section 5.2), covered in full at Level 5.
- **Smart order routing** — the venue-selection logic that complements execution scheduling in fragmented markets (Section 5.3), also a Level 5 topic.
- **Execution alpha** — using short-term order-book signals (order flow imbalance, queue position, micro-price) to opportunistically improve fill prices within an execution schedule, rather than treating execution as purely a cost to minimize.

Professional Applications

- Transaction cost analysis (TCA) — systematically measuring realised execution quality against arrival price and other benchmarks, to evaluate brokers, algorithms, and trading decisions after the fact.
- Realistic backtest cost modelling — building the participation-rate-aware cost models from Section 6.2 into a research pipeline, so strategy evaluation reflects achievable execution rather than an idealized fill price.
- Execution algorithm selection — choosing between TWAP, VWAP, POV, and more sophisticated liquidity-seeking algorithms based on an order's specific urgency, size, and the liquidity profile of the instrument being traded.

Where to Go Next

This module introduced execution from the research side — enough to build realistic cost assumptions and interpret execution quality, without yet implementing a live execution system. The Development curriculum's infrastructure modules cover how to build an execution simulator or connect to a broker's execution algorithms in practice. For the full mathematical treatment of Almgren-Chriss optimal execution, smart order routing, and execution alpha, continue to the Level 5 Execution (Trading Perspective) module, which treats this material as its core subject rather than an introduction.